

Learned Page Replacement for xv6

Training on verified reference streams, and what a kernel can actually use

`traces2 study — report/week_14_update_report.pdf`

Can a learned model replace the kernel's page-replacement policy?

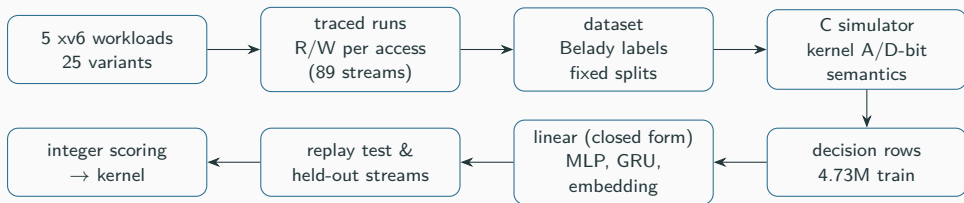
Train on memory workloads, integrate the model into xv6, measure it — a precursor to Linux.

Three constraints shape everything:

1. **No floating point in xv6.** Anything learned must run in integers.
2. **A kernel never sees memory accesses.** The MMU handles loads and stores; the kernel sees faults and the accessed/dirty bits it can read at a scan. Same in Linux for anonymous memory.
3. **Decisions compound.** Policies can only be compared by replaying whole reference streams — all results are fault counts, not prediction accuracy.

So every model is trained on three **feature tiers**: what only an oracle trace has, what a kernel observes, and what a kernel can cheaply book-keep — the first is an *ablation*, not a deployable result.

Pipeline



The workloads

Workload	Models	Why no single heuristic wins
btreebench	B+tree (SQLite pager): insert, lookup, scan, mixed, WAL	lookups favour frequency, inserts recency, scans sequential
kvbench	Zipfian hash table (Redis): YCSB modes A/B/C/D/F, rehash, value sizes	hot set rotates: old-hot pages must go cold
graphbench	BFS + fixed-point PageRank on a scale-free graph	data-dependent frontier; hubs revisited each iteration
sortbench	bottom-up merge sort	deliberately sequential (calibration)
matmulbench	naive vs. blocked matrix multiply	locality contrast, tiny working sets

Collecting a complete, verified reference string

- Every arena access goes through `vmbench_trace_ref(vpn, access)`: one line `R <vpn>` or `W <vpn>` per access — every touch, not just faults. No default access type: an unmarked site does not compile.
- A reference string does not depend on memory size, so each stream is **one run at a generous margin**; capacities are chosen in simulation.
- $25 \text{ variants} \times \text{seeds} = \mathbf{89 \text{ streams}}$ + 22 repeats, 6 parallel lanes. A stream counts only if the run passed and the file matches `TRACEEND refs=N bytes=M` exactly.
- All 111 runs passed; all 22 repeats **byte-identical**.
- Checked against the kernel: FIFO replay reproduces its evictions with 3–5 untraced frames (−0.1%); `W` marks predict its `page_writes`.

The dataset and its splits

Per reference: `vpn`, `write`, `next_use`.

`next_use` = distance to the page's next access — what Belady evicts by. A **label only**, never an input.

held-out: variants never trained on

kv-F, btree-scan, graph-bfs2000, sort-n60000, matmul-128

test: last seed of every other variant

val: second-to-last seed

Split	Streams	References
train	41	45.5M
val	15	23.6M
test	15	23.6M
held-out	18	30.6M

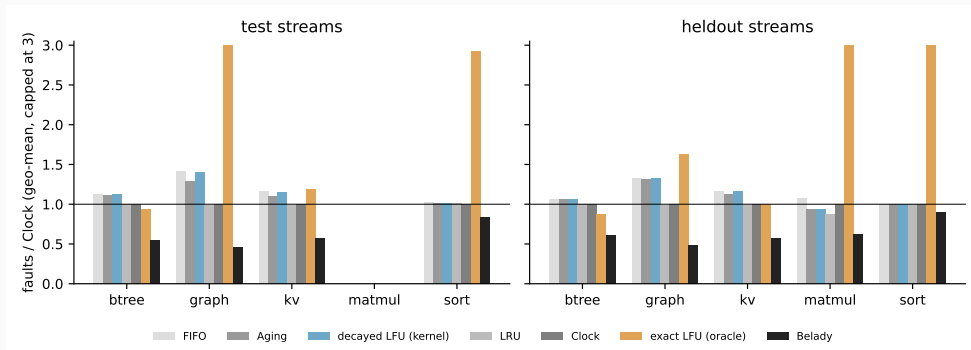
Three feature tiers

Tier	Feature	Definition	Linux analogue
F (oracle)	rec	references since last access	—
	freq	exact access count	—
	sd	distinct pages since last access	—
	wr	fraction of accesses that wrote	—
K (kernel)	ref	accessed bit at this scan	referenced / young bit
	aging	8-bit aging counter (xv6 Aging)	—
	sfreq	scans that found it accessed	DAMON <code>nr_accesses</code> , MGLRU tiers
	idle	scans since last found accessed	MGLRU generations, DAMON age
	age dirty	scans since loaded dirty bit	— PG_dirty
K+ (bookkeeping)	refaults	times evicted and faulted back	mm/workingset.c shadow entries
	rdist	evictions between eviction and refault	

Training on decisions

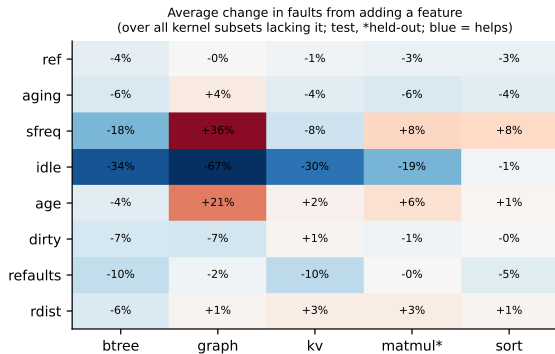
- Replay every train/val stream at 5, 10, 20% of its distinct pages under LRU; at sampled evictions record 32 resident candidates — Belady's choice, the 4 most recently used, and random others — with all 12 features and $\log(1 + \text{distance to next use})$.
- **4.73M rows over 357K eviction decisions**; 0.84M validation rows.
- Models: **linear** (closed form, any subset), **MLP** 2×32 , a **ranking loss** (softmax over one decision's candidates), **GRU** over the last 8 access intervals, **cross-page embedding**.
- Scopes: one model per workload, and one **global** model — the realistic one: a kernel does not know which program is running.
- **Every combination**: $300 \text{ feature sets} \times 6 \text{ scopes} = 28,800$ full replays at 10%.
- **Nested selection on validation**: top-5 subsets per tier $\times$ probation $\{0, 1, 2, 4\}$ at 5/10/20%; only the winner is run on test and held-out.

What xv6 can already do: the classical policies



Faults relative to Clock (xv6's best), geo-mean over streams $\times 5/10/20\%$. Aging $\approx$ FIFO; exact LFU wins on btree and is catastrophic elsewhere; Belady needs 43–54% fewer faults on btree/kv/graph.

What each kernel feature is worth



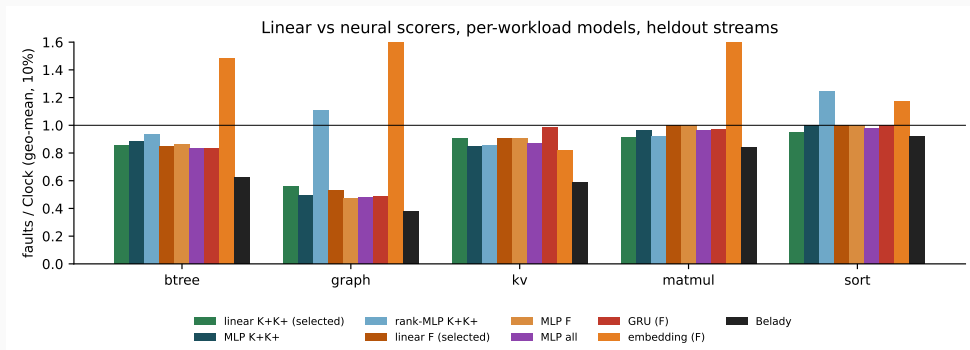
Median change in faults when a feature is added to a subset that lacks it.

- `idle` — scans since last found accessed — dominates: **-71% graph**, **-29% btree**, **-20% kv**.
- That is exactly what Linux's MGLRU generations and DAMON age track.
- Refault count helps btree/kv/sort.

The kernel's own signals are enough

- With only accessed/dirty-bit scan history + refault bookkeeping, the selected policy **beats Clock on every workload**, on unseen seeds *and* unseen variants:
btree -17/ -14%, graph -15/ -20%, kv -10/ -10%, matmul -32%, sort -2/ -7% (test / held-out).
- The oracle features are within $\pm 1.5\%$ of it on btree/graph/kv and behind on matmul and sort:
what a kernel cannot see is not what limits a learned policy here.
- Refault bookkeeping helps in all 9 workload $\times$ split cells.
- Held-out variants generalise as well as unseen seeds.
- **One global model** (the deployable case) still beats Clock in 8 of 9 cells, keeping most of the gain on btree/graph/kv.

Neural, sequence and ranking models



A linear score is competitive with everything: MLP gains on kv and held-out graph only; the oracle GRU adds nothing; the cross-page embedding is catastrophic on btree/graph but best on held-out kv; the ranking loss did not help.

Can it run without floating point?

workload	split	float	int, 4-bit	int, 8-bit	int, 12-bit
btree	test	0.833	0.834	0.833	0.833
btree	heldout	0.863	0.875	0.862	0.863
graph	test	0.845	0.848	0.845	0.845
graph	heldout	0.798	0.812	0.799	0.798
kv	test	0.905	0.905	0.905	0.904
kv	heldout	0.905	0.903	0.904	0.904
matmul	heldout	0.681	0.681	0.681	0.681
sort	test	0.983	0.983	0.983	0.983
sort	heldout	0.933	0.932	0.933	0.933

With 8-bit weights and Q8 features every selected model is within ± 0.002 of float (most identical); 4-bit stays within $\sim 2\%$. The policy runs in xv6 as trained — no floating point.

VM_POLICY_ML: the learned policy in the kernel

- `choose_ml()` in `kernel/vmpage.c`: scan candidates (read/clear PTE_A), integer score $\sum_j q_j X_j$, evict the highest; probation for new pages.
- Features from `kernel/mlfeat.h`, **shared with the simulator**: table `log1p`, Q8, no floating point, no `libgcc`.
- Refault bookkeeping per swap slot (Linux's shadow entry): eviction scan + refault count.
- Weights **per process** (`vmctl(VM_SET_ML_WEIGHTS)`, inherited on fork/exec); `vmrun ml kv kvbench...`
- Passes data-invariance with all five policies (byte-identical data).
- Fixed on the way: LFU livelock (decayed counter); benchmark burn phase gave each policy a different memory size (now always FIFO).

231 runs: 33 test/held-out streams $\times$ 7 policies, 10% memory, kernel counters.

In xv6: faults relative to Clock

Test (unseen seeds)

workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.143	1.116	1.127	0.843	0.823	0.824
graph	2.096	1.585	2.027	0.989	0.614	0.630
kv	1.284	1.107	1.175	0.875	0.871	0.908
sort	1.331	0.988	0.991	0.980	0.960	0.988

Held-out (unseen variants)

workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.087	1.063	1.073	0.893	0.854	0.893
graph	1.289	1.264	1.274	0.535	0.541	0.550
kv	1.312	1.133	1.187	0.878	0.875	0.924
matmul	1.820	0.999	0.999	1.115	0.973	0.999
sort	1.190	0.998	1.006	0.977	0.933	0.998

Per-workload weights **beat Clock on every workload**: btree $-17.7/-14.6\%$, graph $-38.6/-45.9\%$, kv $-12.9/-12.5\%$, sort $-4.0/-6.7\%$, matmul -2.7% . The global model **loses on matmul** ($+11.5\%$).

In xv6: page writes relative to Clock

Test						
workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.058	1.026	1.028	0.952	1.022	1.043
graph	2.533	1.750	2.301	0.515	0.045	0.045
kv	1.992	1.462	1.645	0.737	0.682	0.715
sort	1.148	0.982	0.987	0.965	0.944	0.982

Held-out						
workload	FIFO	Aging	LFU (decayed)	ML global	ML per-workload	ML per-workload, K only
btree	1.069	1.018	1.019	0.984	0.966	0.981
graph	1.375	1.323	1.332	0.395	0.138	0.130
kv	1.221	1.133	1.171	0.878	0.875	0.924
matmul	13.002	0.987	0.987	3.889	0.035	0.987
sort	1.097	0.995	1.010	0.971	0.925	0.995

Graph 95.5% / 86.2% fewer writes, matmul 96.5%, kv 31.8% (test) — the dirty bit at work.

Did the simulator predict the kernel?

workload	split	FIFO	ML global	ML per-workload
btree	test	1.143 → 1.121	0.843 → 0.841	0.823 → 0.804
btree	heldout	1.087 → 1.068	0.893 → 0.893	0.854 → 0.853
graph	test	2.096 → 1.632	0.989 → 0.659	0.614 → 0.625
graph	heldout	1.289 → 1.262	0.535 → 0.515	0.541 → 0.561
kv	test	1.284 → 1.159	0.875 → 0.891	0.871 → 0.889
kv	heldout	1.312 → 1.174	0.878 → 0.901	0.875 → 0.905
matmul	heldout	1.820 → 1.338	1.115 → 1.338	0.973 → 0.913
sort	test	1.331 → 0.976	0.980 → 0.967	0.960 → 0.950
sort	heldout	1.190 → 1.000	0.977 → 0.986	0.933 → 0.951

Kernel → simulator, faults/Clock: **agreement to a few hundredths** for the ML models. FIFO is worse in the kernel (it also evicts code/stack pages). Graph test: one PageRank stream sits where 2 extra frames halve the faults.

The price: victim-selection time

policy	ticks / eviction	candidates / eviction
Clock	35.3	106.0
FIFO	12.6	106.0
Aging	199.6	106.0
LFU (decayed)	199.4	106.0
ML global	411.0	106.0
ML per-workload	451.7	106.0
ML per-workload, K only	400.6	106.0

ML scores every resident page on each eviction: ≈ 43 ticks + 3.9 per page, 8.8–101.2 μ s, 2.4–32.4 $\times$ Clock, 1.8–2.5 $\times$ Aging (means above are over all runs). Small next to a disk read in xv6; a real kernel would score a sample or batch (as MGLRU does).

1. On verified reference streams, a **linear score over 4–5 kernel-observable features** beats xv6's best policy on all five workloads, on unseen seeds and unseen variants — 2–32% fewer faults (10–32% on btree, graph, kv, matmul), up to 66% fewer disk writes.
2. **Oracle information buys almost nothing** over those features; `idle` (the kernel's own recency) carries most of the value.
3. **Bigger models do not pay**: MLP, GRU, cross-page embedding, ranking loss are no better than linear except MLPs on kv and held-out graph.
4. **It works in the kernel**: integer-only `VM_POLICY_ML` beats `Clock` in xv6 on all five workloads (2.7–45.9% fewer faults, up to 96.5% fewer writes), as the simulator predicted; selection costs $2.4\text{--}32.4\times$ `Clock`.

Next steps

1. Cheaper selection: score a sample/batch of candidates instead of all of them.
2. In-kernel runs at 5% and 20% memory (mechanical; hours of QEMU time).
3. Pick weights automatically per program or phase (the global model loses on matmul).
4. Capacity-aware models for graph at 5% and 20%.
5. Port to Linux: `idle`, `sfreq`, `dirty`, `refaults`, `rdist` map to MGLRU, DAMON and workingset signals Linux already keeps.